

操作系统

第一章 绪论（下）

同济大学计算机系

Part 1 深入理解 应用程序 总结、提高

- 应用程序 = 代码 + 数据。
 - 启动前，程序存放在磁盘上，是可执行文件；
 - 启动时，要为其分配内存，存放代码、数据，供CPU取用；
 - 程序执行要用CPU；这块硬件芯片会逐条执行内存中的指令（在程序计数器PC的指引下 取指令，解码，取操作数，运算，存运算结果），直至应用程序运行结束。

∴ 程序运行，需要使用 内存 和 CPU 资源

• 阅读、编写方阵自乘程序，回答以下问题

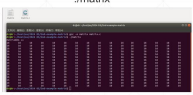
```
#include <stdlib.h>
#include <stdio.h>

#define M 16
int version = 1;
int matrixOriginal[M][M];
int matrixDes[M][M];

void produce ( int row, int column )
{
    int i;
    for (i=0;i<M;i++)
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
}

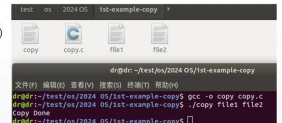
int main()
{
    int i, j;
    for (i=0;i<M;i++)
        for (j=0;j<M;j++)
            produce(i, j);
    printf("matrixDes == \n");
    for (i=0;i<M;i++)
        for (j=0;j<M;j++)
            printf("%d\t", matrixDes[i][j]);
    printf("\n");
}
```

1、matrix程序的代码，包括？
2、指出matrix程序的只读数据、全局变量和局部变量。
3、M是什么？（是编译器使用的符号，不是应用程序的数据）？
请将 M 修改为4096，测速，为什么程序慢那么多？加速它。



应用程序示例 CLI程序copy

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");
    int oldFile = open(argv[1], O_RDONLY);
    int newFile = open(argv[2], O_WRONLY|O_CREAT|S_IRUSR|S_IWUSR);
    char c;
    while( read(oldFile, &c, 1) == 1 )
        write(newFile, &c, 1);
    printf("Copy Done\n");
    close(oldFile);
    close(newFile);
}
```



copy程序一次一字节地将已有文件file1的内容复制到新文件file2

▪ 阅读、编写方阵自乘程序，回答以下问题

```
# include <stdlib.h>
# include <stdio.h>

# define M 16
int version = 1;
int matrixOriginal[M][M];
int matrixDes [M][M];

void produce ( int row, int column )
{
    int i;
    for (i=0;i<M;i++)
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
}

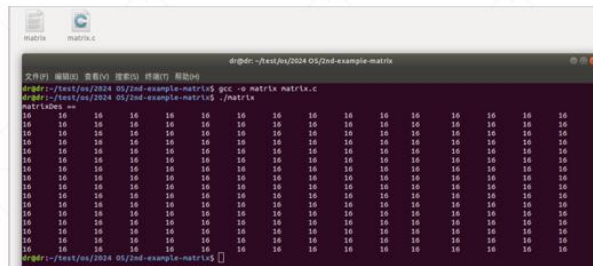
int main()
{
    int i , j;

    for (i=0;i<M;i++)
        for (j=0;j<M;j++) {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
        }

    for (i=0;i<M;i++)
        for (j=0;j<M;j++)
            produce(i , j);

    printf("matrixDes == \n");

    for (i=0;i<M;i++) {
        for (j=0;j<M;j++)
            printf("%d\t", matrixDes[i][j]);
        printf("\n");
    }
}
```



- 1、matrix程序的代码，包括？
- 2、指出matrix程序的只读数据，全局变量 和 局部变量。
- 3、M是什么（是编译器使用的符号，不是应用程序的数据）？请将 M 修改为4096，测速。为什么程序慢那么多？加速它。

只计算，不IO。。。程序运行时不使用外设，不会申请额外资源

应用程序示例 CLI程序copy

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

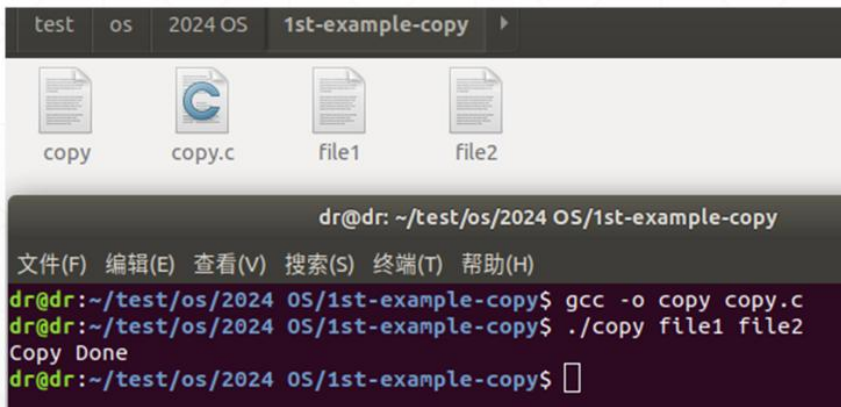
    int oldFile = open(argv[1],O_RDONLY);
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
    while( read(oldFile,&c,1) == 1 )
        write(newFile,&c,1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdlib.h>
#include <stdio.h>



```
dr@dr: ~/test/os/2024 OS/1st-example-copy
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
dr@dr:~/test/os/2024 OS/1st-example-copy$ gcc -o copy copy.c
dr@dr:~/test/os/2024 OS/1st-example-copy$ ./copy file1 file2
Copy Done
dr@dr:~/test/os/2024 OS/1st-example-copy$
```

copy程序一次一字节地将已有文件file1的内容复制到新文件file2

只IO，不计算。。。程序运行时会频繁使用外设，申请使用硬盘资源

旁白 应用程序类型：CPU密集型 和 IO密集型

CPU密集型 (CPU bound)

• 阅读、编写方阵自乘程序，回答以下问题

```
#include <stdlib.h>
#include <stdio.h>

#define M 16
int version = 1;
int matrixOriginal[M][M];
int matrixDes[M][M];

void produce ( int row, int column )
{
    int i;
    for (i=0;i<M;i++)
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
}

int main()
{
    int i, j;
    for (i=0;i<M;i++)
        for (j=0;j<M;j++)
            produce(i, j);
    for (i=0;i<M;i++)
        for (j=0;j<M;j++) {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
        }
}
```

1、matrix程序的代码，包括？
2、指出matrix程序的只读数据、全局变量和局部变量。
3、M是什么（是编译器使用的符号，不是应用程序的数据）？
请将M修改为4096，测速。为什么程序慢那么多？加速它。



操作系统(拔尖班 / 国豪 2025) 同济大学计算机系 邓蓉 15

IO密集型 (CPU bound)

应用程序示例 CLI程序copy

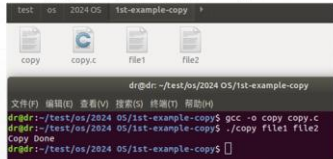
```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY);
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
    while( read(oldFile,&c,1) == 1 )
        write(newFile,&c,1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```



copy程序一次一字节地将已有文件file1的内容复制到新文件file2

操作系统(拔尖班 / 国豪 2025) 同济大学计算机系 邓蓉 10

更多的是混合型

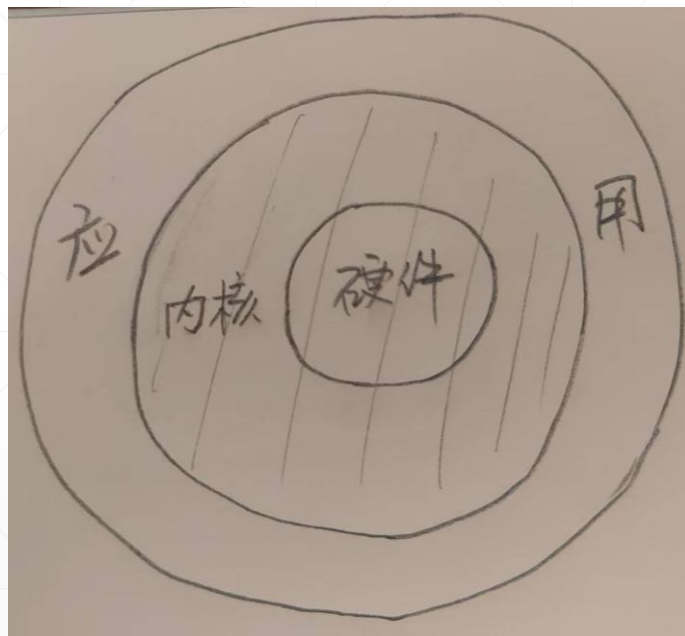
哔哩哔哩：
视频下载，IO密集阶段
解码播放，CPU密集阶段

Word：
打字，？
拼写检查，？
.....

Part 2 计算机系统

- 计算机系统用来执行程序，包括硬件和操作系统内核。

用系统调用的方式向应用程序提供服务（资源访问、数据传输和运行控制）。



- 每个计算机系统配有一个内核，以内核名称命名。所以，管理系统资源的是windows内核，这个系统就是windows系统；~Linux内核，~~~Linux系统。

系统调用 是 计算机系统的应用程序接口

```
./copy file1 file2
```

- 应用程序执行系统调用，请求内核服务。

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY); ← 请求读文件file1
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);
                  ↑ 请求创建新文件，并且写入
    char c;
    while( read(oldFile,&c,1) == 1 ) ← 请求读文件，一个字节
        write(newFile,&c,1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile); ← 通知内核，文件访问完毕
}
```

系统调用，内核的响应（例）

```
int main( int argc, char* argv[] )  
{  
    if(argc!=3)  
        printf("Usage: copy oldfile newfile\n");
```

```
    int oldFile = open(argv[1],O_RDONLY);  
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);
```

搜索目标文件file1，检测用户的文件访问权限（可以读？）
用户可不可以在父目录中创建新文件

```
    char c;  
    while( read(oldFile,&c,1) == 1 )  
        write(newFile,&c,1);
```

内核跟踪文件访问状态，第1次read返回0#字节，第2次read返回1#字节。。。

内核记录为文件分配的磁盘扇区号n1、n2、n3, 第1次read读n1扇区、0#字节，第2次read读n1扇区、1#字节..... 第512次read读n2扇区、0#字节（扇区，512字节）

```
    printf("Copy Done\n");
```

```
    close(oldFile);  
    close(newFile);  
}
```

内核释放用来跟踪文件访问状态的数据结构



系统调用屏蔽资源访问、硬件驱动细节，为程序员提供使用方便、语义精确、性能卓越的内核接口

请求

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY); // 请求读文件file1
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);
                // 请求创建文件file2，要写这个文件
    char c;
    while( read(oldFile,&c,1) == 1 ) // 请求读文件，一个字节
        write(newFile,&c,1);

    printf("Copy Done\n");

    close(oldFile); // 通知内核，文件访问完毕
    close(newFile);
}
```

响应

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY); // 找到目标文件file1，检测用户的文件访问权限（可以读？）
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR); // 用户可不可以不在父目录中创建新文件

    char c;
    while( read(oldFile,&c,1) == 1 ) // 内核跟踪文件访问状态，第一次read返回0#字节，第二次read返回1#字节。。。
        write(newFile,&c,1); // 内核记录为文件分配的磁盘扇区n1、n2、n3.....，第1次read读n1扇区、0#字节，第2次read读n1扇区、1#字节.....
                                // 第512次read读n2扇区、0#字节.....（扇区，512字节）

    printf("Copy Done\n");

    close(oldFile); // 内核释放用来跟踪文件访问状态的数据结构.....
    close(newFile);
}
```

每个系统调用对应一个内核函数（系统调用入口函数）。系统调用是应用程序调用内核函数。

应用程序

copy, shell,

内 核

Sys_Open, Sys_Read, Sys_Write, Sys_Close 64个系统调用入口函数

系统调用入口函数位于内核最外层，屏蔽资源访问和系统控制细节，简化程序开发流程，提高系统效率和稳定性。系统程序员面向系统调用编程，使用内核抽象出来的虚拟机，精细控制应用系统执行过程，提高运行效率，降低资源消耗。

不同操作系统，不同的系统调用。在Ubuntu（Linux）系统上开发的应用程序，windows系统跑不起来。

为单道系统服务的操作系统

- 单道系统：内存中只有一个程序，这个运行中的程序独占全部系统资源。运行完毕，启动下一个程序。
- 为单道系统服务的操作系统包括 2 个程序：
 - 内核： 驱动硬件，管理磁盘上的文件，实现系统调用入口函数。
 - 命令行界面（用户接口）： 解析用户输入的命令行，启动应用程序。

内核结构

系统调用入口函数

其它内核函数 &
内核数据结构

（静态数据、动态数据，它们是内核的静态全局变量、堆变量 和 局部变量）



例：DOS (Disk Operating System)

DOS是运行在PC (Personal Computer, 个人计算机) 上的操作系统, 任何时刻系统中只运行一个程序, 独占所有系统资源。运行结束后, 系统才能够执行另一个程序。

DOS操作系统的组成

- 一个命令行界面 cmd
 - 供用户输入DOS命令。
- 一个DOS内核, 包括
 - 一个文件系统。包括一组设计精巧的内核数据结构, 登记磁盘空间的使用状态: 包含哪些文件夹, 每个文件夹有哪些文件。为每个文件分配的所有扇区号。
 - 一个供应用程序使用的系统调用接口, 用来接收、响应 资源访问请求。
 - 一组质量上乘的内核函数, 用来实现系统调用, 驱动CPU和所有外部设备, 访问软件资源。

```
命令提示符 - date
Microsoft Windows [版本 10.0.22631.4037]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\Pretty-Girl>ping 127.0.0.1

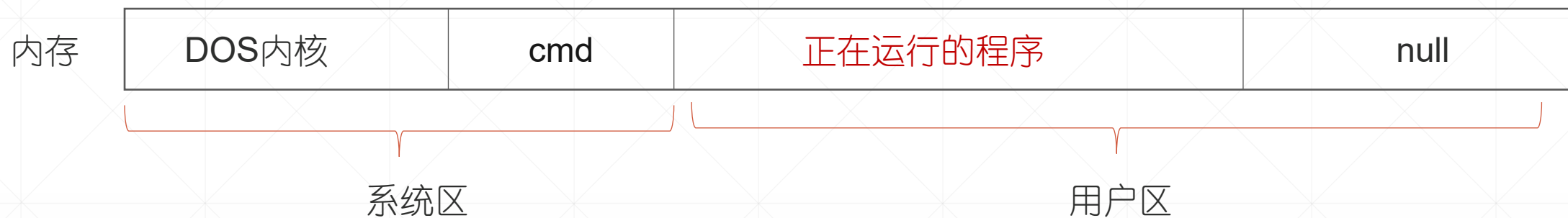
正在 Ping 127.0.0.1 具有 32 字节的数据:
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128

127.0.0.1 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 0ms, 平均 = 0ms

C:\Users\Pretty-Girl>date
当前日期: 2024/08/30 周五
输入新日期: (年月日) |
```

单道, 所以DOS不需要资源调度, 不需要安排程序的执行次序

DOS系统的内存



多道系统

- 多道系统：允许内存并发多个程序。

内存

操作系统内核	程序1 (shell)	null	程序2	程序3	null
--------	-------------	------	-----	-----	------

- 这些程序**共享**系统资源。
- 得到资源的，前进；没有得到资源的，等待；运行结束的，终止、释放内存资源。
这种走走停停的运行方式叫**异步**。
- 避免程序相互干扰，系统要对它们进行**隔离**。

内核对多道系统的支持 1、隔离

- 禁止正在执行的程序访问分配给其它程序的内存单元。



CPU正在执行程序1时，只能从这个区域取指令，只能读写存放在这个区域的变量

操作系统

借助CPU提供的异常机制隔离并发的应用程序，杀死越界程序。

内核对多道系统的支持

2、单核系统中的并发

- **并发**，同时执行。

内存	操作系统内核	程序1 (shell)	null	程序2	程序3	null
----	--------	-------------	------	-----	-----	------

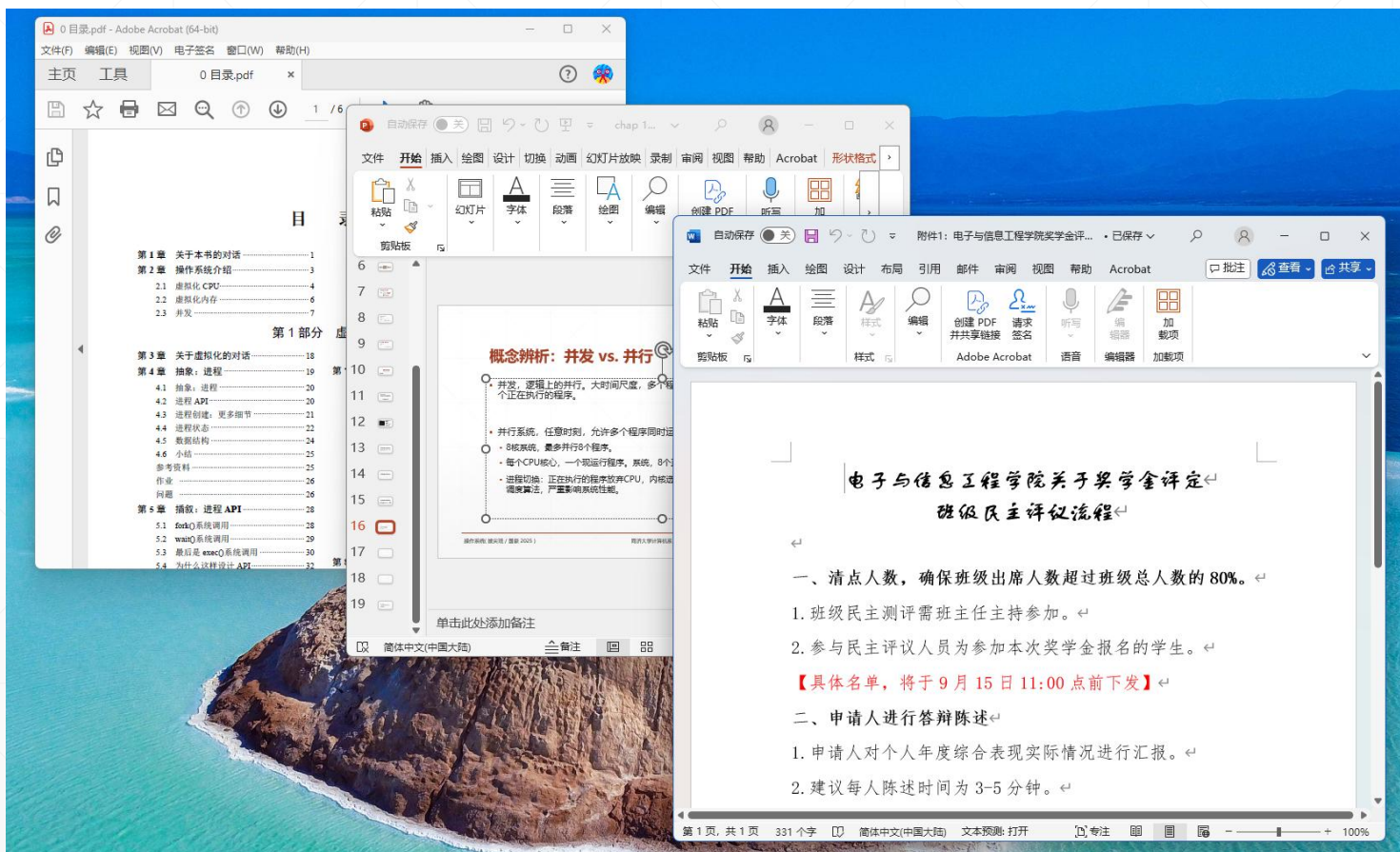
硬件约束：任意时刻，一个CPU（核心，core）只能运行一个程序。

单核系统怎样实现并发？多个程序轮流使用CPU。运行一段时间后，换个程序跑跑；**进程切换**。

支持并发，内核必需为每个程序设置 **进程控制块（Process Control Block, 简称PCB）**，这个内核数据结构登记程序执行状态。

正在执行的程序1让出CPU的时候，内核需要在PCB中登记断点（下条指令地址）。。。未来恢复运行，从PCB中取断点→PC寄存器。

并发 3 个程序*



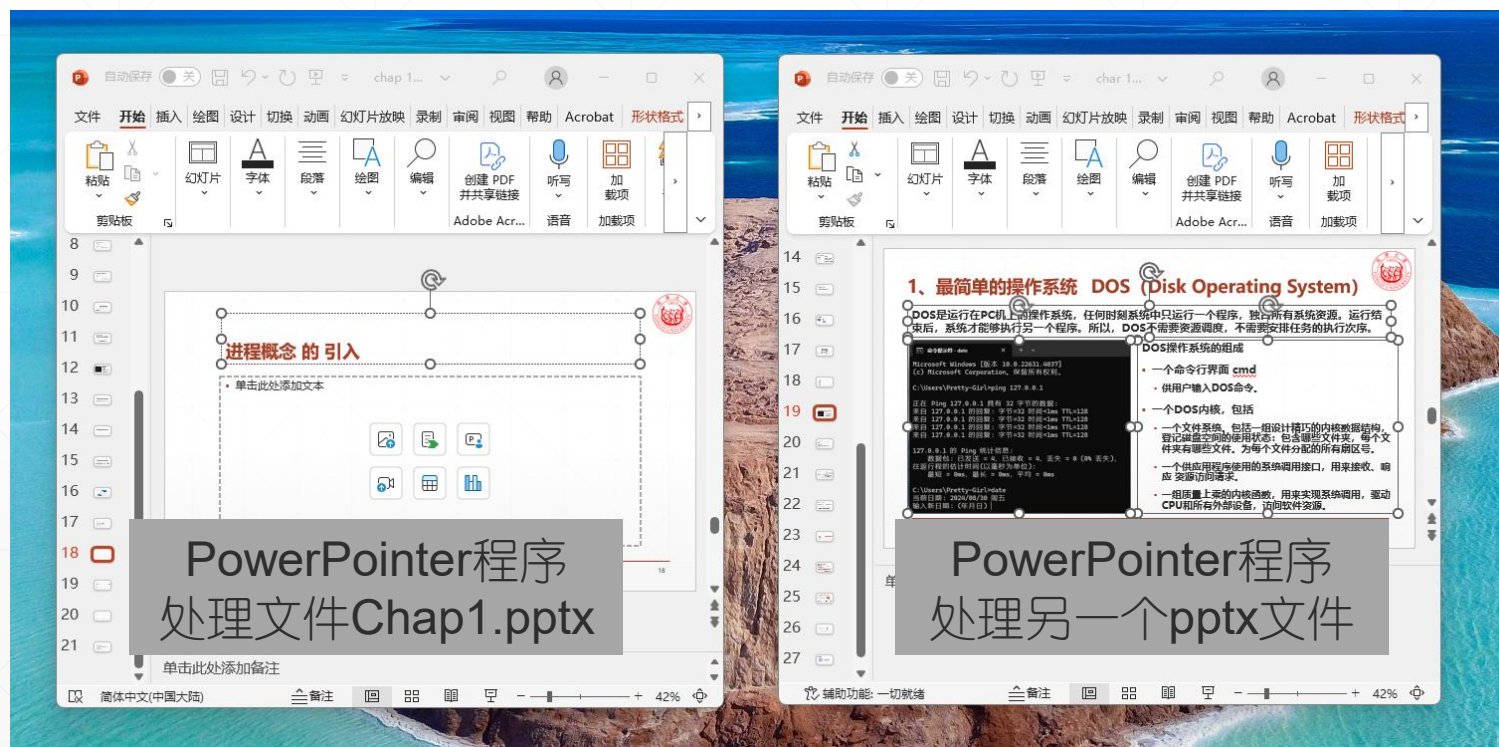
其实是4个

概念辨析：并发 vs. 并行

- 并发，逻辑上的并行。大时间尺度，多个程序同时向前推进。但任意时刻，只有一个正在执行的程序。
- 并行系统，任意时刻，允许多个程序同时运行。举例：
 - 8核系统，最多并行8个程序。
 - 每个CPU核心，一个现运行程序。系统，8个正在执行的程序。
 - 进程切换：正在执行的程序放弃CPU，内核选中另一个程序（**调度**），恢复其执行状态。调度算法，严重影响系统性能。

进程概念的引入

- 需要一个机制，区分同一个程序的多个并发执行实例。这个机制，叫做进程（Process）。



定义

- 进程是程序的一次执行过程
- 进程是运行中的一个程序实例
- 进程 = 代码 + 数据 + PCB
- 需要运行时，系统创建一个新进程，让这个进程承担执行这个应用程序的任务。程序跑完了，进程就没用了，exit终止自己。自诞生至终止，进程生命周期
- PCB是进程存在的唯一标识
- 进程，相较其执行的应用程序，后者是菜谱，前者是一次完整的烹饪过程
- 进程并发，进程调度，为进程分配资源（比如内存空间）
- 本课程，进程以所执行的程序命名。前页ppt，第一个powerpointer进程，第二个powerpointer进程。。。



熟悉的多道系统

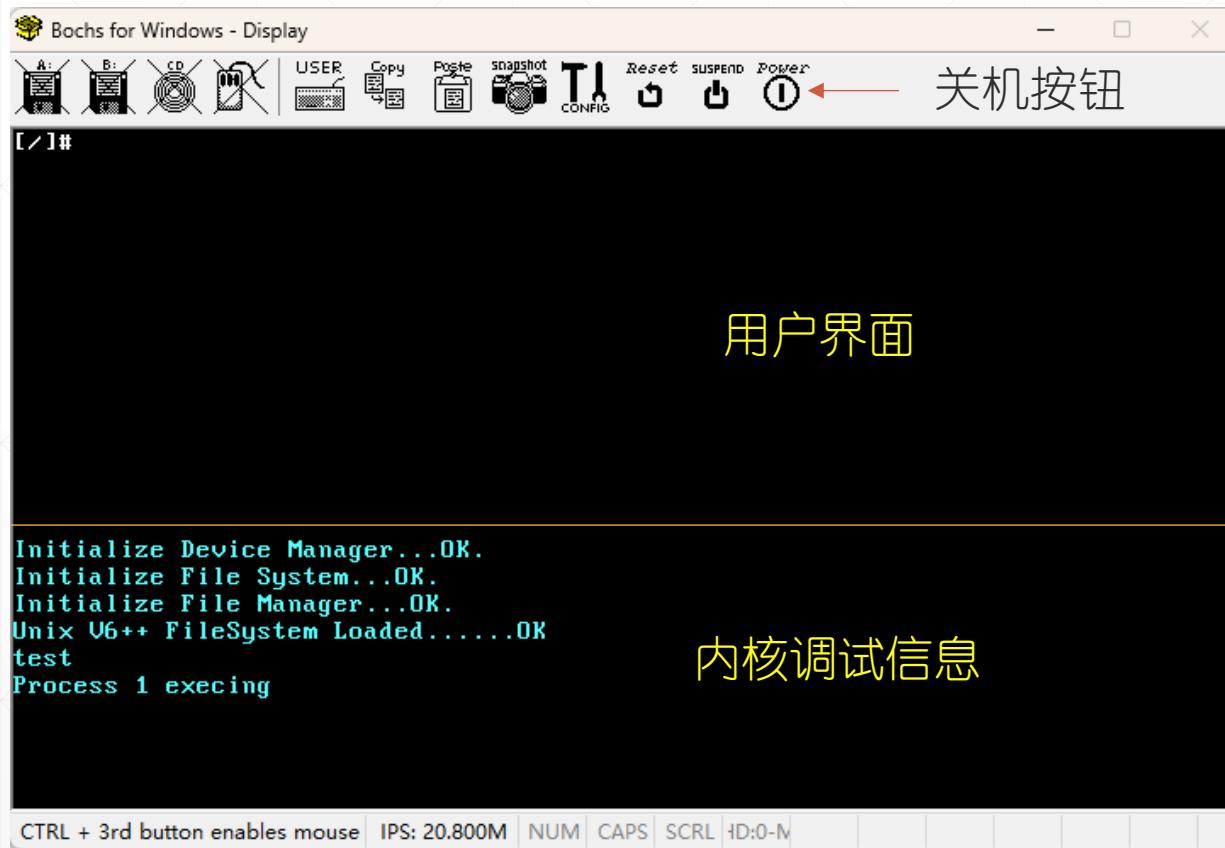
- 运行在PC机上的 Windows、Linux (Ubuntu) 、 macOS (Apple) 、 Unix V6+ +
- 移动操作系统 iOS, 安卓
- 运行在服务器上的 Linux (Ubuntu, CentOS (Community Enterprise Operating System))
- 更多不常见的操作系统

单用户
操作系统

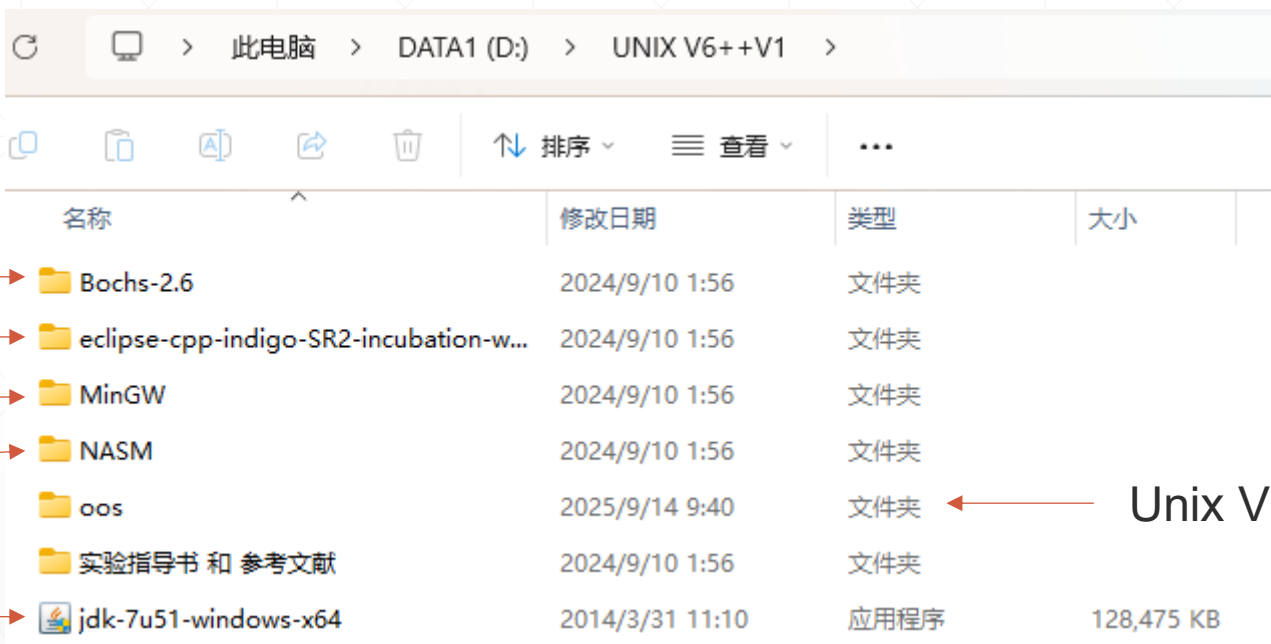
多用户
操作系统

Part 3、Unix V6++系统

- 多道系统
- 单用户
- 字符界面
- 运行在Bochs虚拟机上
 - CPU型号: Intel 386
 - 内存容量: 32M字节
 - 一个硬盘
 - 一个键盘, 一个显示器
- 开发环境: Eclipse 或 VS Code

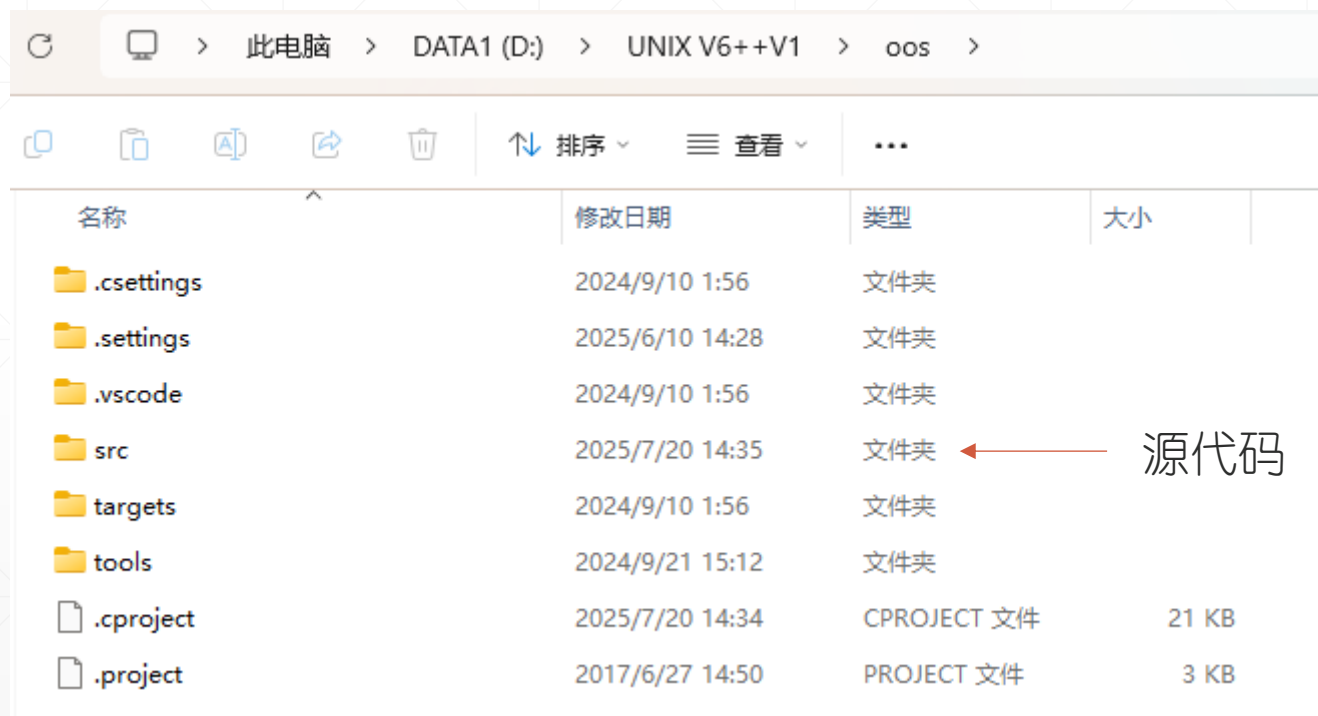


Bochs虚拟机
集成开发环境
编译、链接 内核、应用程序
汇编引导程序
eclipse运行在Java虚拟机上



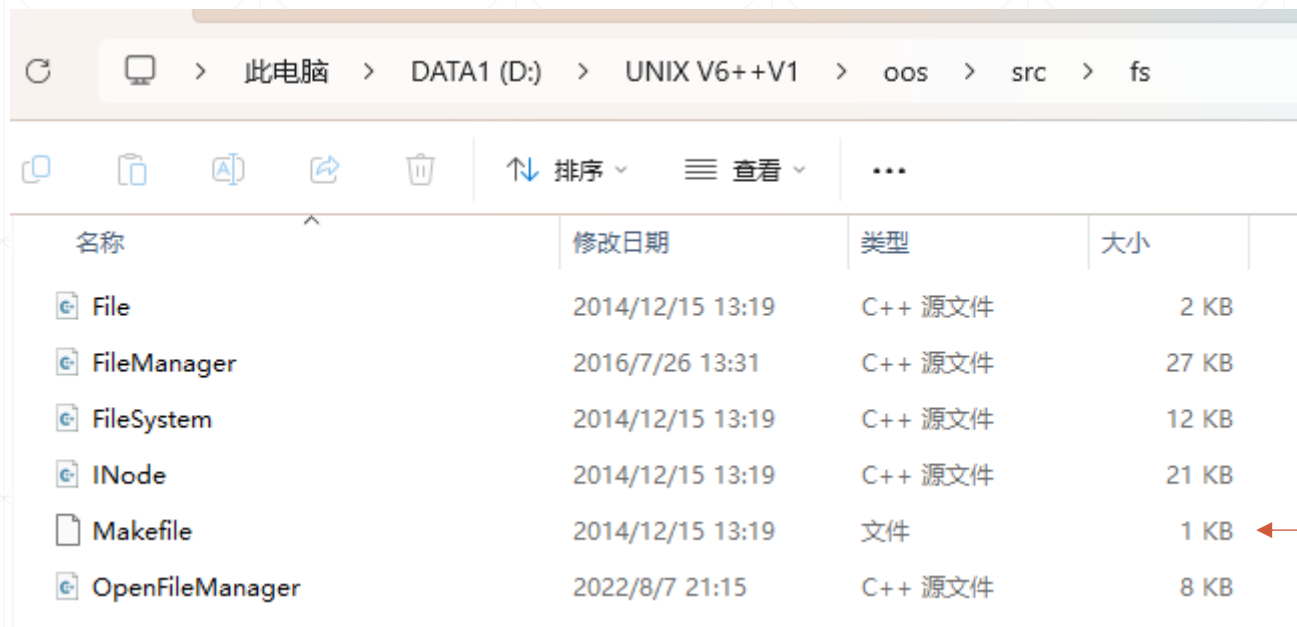
名称	修改日期	类型	大小
Bochs-2.6	2024/9/10 1:56	文件夹	
eclipse-cpp-indigo-SR2-incubation-w...	2024/9/10 1:56	文件夹	
MinGW	2024/9/10 1:56	文件夹	
NASM	2024/9/10 1:56	文件夹	
oos	2025/9/14 9:40	文件夹	
实验指导书 和 参考文献	2024/9/10 1:56	文件夹	
jdk-7u51-windows-x64	2014/3/31 11:10	应用程序	128,475 KB

Unix V6++系统





此电脑 > DATA1 (D:) > UNIX V6++V1 > oos > src >				
排序 查看 ...				
名称	修改日期	类型	大小	
boot	2025/7/20 14:35	文件夹		boot, 引导程序
dev	2025/7/20 14:35	文件夹		shell, 命令行界面
fs	2025/7/20 14:35	文件夹		lib, 函数库
include	2025/7/20 14:35	文件夹		program, 应用程序
interrupt	2025/7/20 14:35	文件夹		其余, 所有.cpp链接成内核, kernel
kernel	2025/7/20 14:35	文件夹		dev, 外设管理模块
lib	2025/7/20 14:35	文件夹		fs, 文件管理模块
machine	2025/7/20 14:35	文件夹		mm, 内存管理模块
mm	2025/7/20 14:35	文件夹		proc, 进程管理模块
pe	2025/7/20 14:35	文件夹		tty, 终端 (键盘、显示器)
proc	2025/7/20 14:35	文件夹		interrupt, 中断处理模块 (系统调用入口函数.....)
program	2025/7/20 14:35	文件夹		kernel, 内核初始化时使用的函数 & 内核工具函数
shell	2025/7/20 14:35	文件夹		machine, 管理、驱动硬件资源
test	2025/7/20 14:35	文件夹		pe, 解析应用程序的程序头
tty	2025/7/20 14:35	文件夹		
Link.ld	2014/12/15 13:19	LD 文件	1 KB	} Unix V6++的编译、链接规则
Makefile	2022/8/8 16:36	文件	3 KB	
Makefile.inc	2022/8/21 16:05	INC 文件	1 KB	



这个文件夹中，所有.cpp的编译规则

Unix V6++的静态链接库





Unix V6++的应用程序

```
[/]#echo Hello Unix
Hello Unix
[/]#date
18-Sep-2025 0:28:45(THUR)
[/]#_

Process 2 is exiting
end sleep
Process 2 (Status:5) end wait
Process 1 finding dead son. They are Process 3 (Status:3) wait
ess Exit! Process 3 execing
regs->eax = 4294967294 , u.u_error = 2
Process 3 execing
Process 3 is exiting
end sleep
Process 3 (Status:5) end wait

CTRL + 3rd button enables mouse IPS: 22.400M NUM CAPS SCRL ID:0-IV
```